Tutoría de Programación Orientada a Objetos

Conceptos sencillos · UML paso a paso · Ejercicios para practicar

Cipriano Rivera Escobar

Fundamentos de Informática · Universidad Nacional de Costa Rica

II Ciclo 2026

¿Qué vamos a aprender?

- Qué son una clase, un objeto, un atributo y un método.
- Cómo proteger datos con private y usar public.
- Para qué sirven get y set.
- Cómo funcionan los constructores y sus parámetros.
- Cómo dibujar y leer un diagrama UML.
- Cómo aplicar abstracción y encapsulamiento.

¿Qué es la programación orientada a objetos?

Idea principal

Es una forma de organizar un programa usando objetos que guardan información y realizan acciones.

Ejemplo cotidiano

En una escuela podemos representar estudiantes, cursos y profesores mediante objetos.

Clase y objeto: ¿cuál es la diferencia?

CLASE = molde

La clase Estudiante describe qué información y acciones tendrá cualquier estudiante.

OBJETO = caso concreto

Ana y Luis son dos objetos de Estudiante. Comparten la estructura, pero sus datos son distintos.

Pregunta: ¿dos estudiantes necesitan dos clases diferentes?

Atributos y métodos

Atributos: ¿qué tiene?

Son los datos que guardamos.

Ejemplo: nombre, edad, nota.

Métodos: ¿qué hace?

Son acciones o consultas.

Ejemplo: estudiar(), mostrarNota().

Truco: atributo = característica · método = acción

Encapsulamiento: private y public

private

Protege los atributos para que no se cambien directamente desde fuera de la clase.

public

Permite utilizar los métodos que la clase ofrece para consultar o modificar datos.

Ejemplo: la edad es privada y se modifica mediante un método que la valide.

Métodos get y set

GET = consultar

getEdad() permite conocer la edad actual de un estudiante. Normalmente devuelve un valor.

SET = modificar

setEdad(18) permite cambiar la edad. Puede comprobar que el dato sea válido.

No todos los atributos necesitan obligatoriamente get y set.

Constructores: ¿para qué sirven?

Constructor sin parámetros

Se ejecuta al crear un objeto y puede asignar valores iniciales.

Ejemplo: edad inicial = 0.

Constructor con parámetros

Recibe datos al crear el objeto.

Ejemplo: nombre = «Ana» y edad = 18.

El constructor tiene el mismo nombre de la clase y no lleva tipo de retorno.

¿Qué son los parámetros?

Parámetro

Es el dato que un método o constructor espera recibir.

Ejemplo: nombre y edad.

Argumento

Es el valor concreto que entregamos cuando lo usamos.

Ejemplo: «Ana» y 18.

Un constructor puede tener cero, uno o varios parámetros.

Tipos de constructores

- Por defecto: permite crear el objeto sin pasar argumentos.
- Parametrizado: recibe datos para inicializar los atributos.
- De copia: crea un objeto a partir de otro de la misma clase.
- Lista de inicialización: inicializa atributos antes del cuerpo del constructor.

UML: cómo se escriben los atributos

Formato

visibilidad nombre : tipo

- edad : int
- nombre : string

Símbolos

- \- privado
- \+ público
- \# protegido

Tipos habituales: int, double, bool y string.

UML: cómo se escriben los métodos

Formato

visibilidad nombre(parámetro : tipo) : retorno

Ejemplos

+ getNombre() : string

+ setNombre(nuevo : string) : void

+ mostrar() : void

UML: cómo se escribe un constructor

Regla

Se escribe con el mismo nombre de la clase. No lleva tipo de retorno.

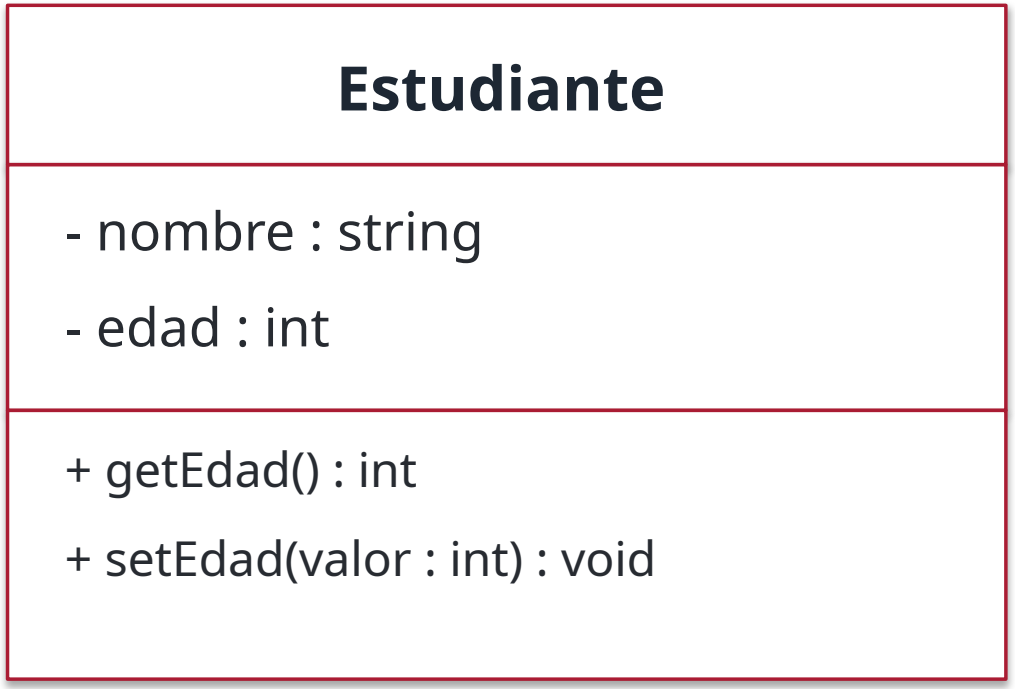
Ejemplos para la clase Estudiante

+ Estudiante()

+ Estudiante(nombre : string, edad : int)

UML: primero aprende las tres partes

Un diagrama de clase UML es un cuadro dividido en tres secciones.



1. Nombre de la clase

2. Atributos y tipos

3. Métodos y retornos

Dos ideas clave: abstracción y encapsulamiento

Abstracción

Quedarnos con lo importante del objeto para el problema.

Encapsulamiento

Proteger los datos y permitir un acceso controlado.

Ejemplo de abstracción

En Estudiante guardamos solo los datos útiles para matrícula.

Ejemplo de encapsulamiento

La nota es privada y se modifica mediante un método.

Abstracción y encapsulamiento: ejemplos

- Abstracción: de Persona guardamos nombre y edad, no todos sus detalles.
- Encapsulamiento: una nota privada se cambia con un método válido.
- Ejemplo de abstracción: Docente y Estudiante pueden especializar a Persona.
- Ejemplo de encapsulamiento: mostrarInfo() puede dar resultados distintos según el tipo.

Ejercicio 1 · Persona

PRÁCTICA · FUNDAMENTOS DE INFORMÁTICA

Enunciado

Crea una clase Persona que guarde nombre, edad e identificación.

Lo que debes hacer

1. Dibuja el UML con los atributos privados.
2. Agrega los métodos get y set de cada atributo.
3. Crea un constructor y un método toString().
4. Crea un objeto en main y muestra sus datos.

Ejercicio 2 · Estudiante

PRÁCTICA · FUNDAMENTOS DE INFORMÁTICA

Enunciado

Crea una clase Estudiante que guarde nombre, carné y nota.

Lo que debes hacer

1. Diseña el UML de la clase.
2. Agrega los métodos get y set.
3. Crea un constructor que reciba los tres datos.
4. Crea un estudiante en main y muestra su información.

Práctica #1 · Mascotas.sa

PRÁCTICA · FUNDAMENTOS DE INFORMÁTICA

Enunciado

Una veterinaria necesita registrar el tipo, raza, edad y motivo de consulta de cada mascota.

Lo que debes hacer

1. Diseña el UML de Mascota con esos cuatro atributos.
2. Agrega métodos get y set y un método para mostrar sus datos.
3. Crea la clase en Visual Studio.
4. Crea una mascota en main y muestra su información.

Práctica #2 · Persona y meses

PRÁCTICA · FUNDAMENTOS DE INFORMÁTICA

Enunciado

Una persona tiene edad, nombre e ID. El programa debe registrar y mostrar su información.

Lo que debes hacer

1. Crea Persona con get, set y toString().
2. Solicita los datos con cin y cout y crea el objeto.
3. Agrega Mes(int mes), que devuelva el nombre del mes usando switch.
4. Prueba el método con un número entre 1 y 12.